

Shane Schisler

Forest Hill, MD 21050

shane.schisler@gmail.com | [linkedin.com/in/shaneschisler](https://www.linkedin.com/in/shaneschisler) | github.com/seschis

[Date]

[Company]

[City, State]

Re: [Position]

Dear [Name / Hiring Manager],

The strongest security engineers I know share one trait: they have spent enough time at the low level — kernels, hypervisors, machine code — that when something new and powerful arrives, their first question is "where does it break, and how do we prove it doesn't?" That question has shaped twenty-five years of my career, and it is exactly the question you need as AI agents start writing, reviewing, and operating production software.

I am a Distinguished Engineer at Contrast Security, where my work centers on the intersection of my whole background: building the tooling that makes AI agents do security work reliably. I architected and built an agentic security-testing tool in Go that stands up arbitrary customer applications, discovers their services, generates the Docker/Compose infrastructure, injects the Contrast runtime agent, and drives a multi-turn LLM loop until every API endpoint is proven reachable. It is hardened for production — per-run spend caps, token-budget scheduling, stuck-loop detection, content-hash caching — and benchmarked by an eval harness that exercises agent behavior against OWASP vulnerable-application targets. Before that work, I architected a petabyte-scale S3 data lake on Iceberg, led the design of the Kafka/Flink event-driven architecture that earned C-suite investment, and cut Java agent installation friction by 90% with a Linux kernel module.

The arc of my career runs from the ground up: kernel drivers and 802.11 security research at Booz Allen; hypervisor introspection and firmware reverse engineering at G2; and a Type-1 hypervisor written from scratch in C at Alithix, where I also authored the technical volume that won a multi-year, multi-million-dollar government contract by out-scoring the incumbent. That depth is why I can look at an LLM agent loop and know exactly where the failure modes live: prompt injection, nondeterministic outputs, cost runaways, and the subtle case where an agent is confidently wrong.

Outside day-to-day work, I build and publish. Harmonia is a multi-LLM triage engine where a panel of models — with persona-driven judge panels on ties — votes on security scanner findings and scores them with CVSS 4.0 (published via Homebrew). dbi-research is a proof of concept for taint-tracking data through a compiled Go binary with Intel Pin, with no source-level hooks. I have reverse engineered hardware at the level of patching OpenOCD to debug an obscure Cortex-M0 MCU on a commercial battery management system. And I write publicly on the governance of AI code security, including a three-tier model for where AI coding agents get security right by default, and a case for importing clinical-trial blinding rigor into AI security evaluations.

What I bring to a team is a working answer to the questions most organizations have not yet answered: where can an LLM be trusted, how do you prove it, how do you bound its cost, and how do you make

the whole thing production-grade? I have run that loop end to end — architecture, systems programming, evaluation, and the organizational alignment to get it funded.

I would welcome the chance to discuss how that experience fits your team. Thank you for your time and consideration.

Sincerely,
Shane Schisler